Jakarta Persistence Applied 온라인 부록

SQL Server FutonManufacturing Persistence

권진

Version 1.0: Jakarta Persistence 3.2 판

Table of Contents

SQL Server FutonManufacturing Persistence 1
sqlserver-samples-persistence-futonmanufacturing 2
FutonManufacturing 3
Attribute Mappings. 25
Exercises 29

SQL Server FutonManufacturing Persistence

이 장에서는 SQL Server 의 예제 데이터베이스 가운데 FutonManufacturing 을 매핑한다. 요(futon)를 만들어 파는 회사의 데이터베이스로, 품목과 여러 단계의 자재 명세, 창고의 재고, 구매와 품질 검사, 생산, 판매를 담는다.

프로젝트의 구성과 매핑의 규칙, 행의 식별자, SQL Server 의 형, 매핑의 테스트는 본서의 SQL Server Sample Databases Persistence 장에서 보았다. 이 장은 테이블을 하나씩 살펴보지 않는다. 먼저 FutonManufacturing 을 매핑한 `sqlserver-samples-persistence-futonmanufacturing` 모듈의 구성을 살펴본다. 다음으로 데이터베이스의 테이블과 뷰, 그 관계와 행 수를 정리하고, 이 데이터베이스가 매핑에 더하는 것을 차례로 분석한다. 같은 테이블을 두 번 참조하는 자재 명세, Jakarta Persistence 에는 없는 재귀 쿼리, 엔티티가 된 뷰와 되지 못한 뷰다. 그다음 컬럼을 속성에 어떻게 매핑했는지 요약하고, 소수의 정밀도와 이름을 살펴본다. 마지막으로 생산 계획 담당자, 구매 담당자, 영업 관리자가 물을 만한 연습문제를 푼다.

sqlserver-samples-persistence-futonmanufacturing

`sqlserver-samples-persistence-futonmanufacturing` 은 FutonManufacturing 을 Jakarta Persistence 로 매핑한 모듈이다. FutonManufacturing 이 어떤 데이터베이스인지는 다음 절인 FutonManufacturing 에서 살펴보고, 이 절에서는 모듈이 어떻게 구성되어 있는지를 먼저 본다.

모듈의 구조는 본서의 SQL Server Sample Databases Persistence 장에서 본 AdventureWorks 의 모듈과 같다.

매핑한 클래스와 테스트

모듈의 패키지는 스키마마다 나뉘지만, FutonManufacturing 의 스키마는 `dbo` 하나뿐이다. `src/main` 의 패키지 `com.github.jinahya.sqlserver.sample.persistence.futonmanufacturing.mapped.dbo` 에는 테이블과 뷰마다 매핑된 슈퍼클래스가 하나씩, 모두 72 개 있다. 복합 키가 없으므로 식별자 클래스는 없다.

`src/test` 의 패키지 `com.github.jinahya.sqlserver.sample.persistence.futonmanufacturing.dbo` 에는 엔티티 50 개와 엔티티가 아닌 클래스 22 개가 있다. 엔티티는 테이블 32 개와 뷰 18 개를, 엔티티가 아닌 클래스는 나머지 뷰 22 개를 매핑한다. 영속성 유닛 `futonmanufacturing-it` 에는 엔티티 50 개가 기재되어 있다. 클래스마다 `*_PersistenceIT` 가 하나씩 있어 클래스의 통합 테스트는 194 개이고, 연습문제의 통합 테스트 28 개와 합한 222 개 가운데 36 개는 읽을 행이 없거나 확인할 관계가 없어 건너뛴다.

클래스의 이름은 테이블의 이름을 따르되, 복수형인 `Items` 만 `Item` 으로 했다. 뷰의 클래스도 같은 규칙을 따라, 뷰의 이름에서 `vw_` 를 떼고 복수형은 단수형으로 했다(`vw_ItemsBelowReorderPoint` 는 `ItemBelowReorderPoint`). `vw_BOMExplosion` 은 `BomExplosion`, `vw_Sales_ByChannel` 은 `SalesByChannel` 다. 엔티티의 이름에는 본서 SQL Server Sample Databases Persistence 장의 매핑의 규칙 에서 본 대로 스키마가 붙으므로, 이 장의 Query Language 는 `DboItem`, `DboBillOfMaterials` 같은 이름을 쓴다.

FutonManufacturing

FutonManufacturing 은 Microsoft 의 `sql-server-samples` 저장소에 새로 더해진 예제 데이터베이스다.[1] 스키마를 만드는 스크립트의 첫 주석은 이 데이터베이스를 여러 단계의 자재 명세와 재고, 생산, 판매를 관리하는 요 제조 회사의 데이터베이스라고 소개한다. AdventureWorks 나 WideWorldImporters 와 달리 백업 파일이 아니라 SQL 스크립트로 배포되고, 데이터도 스크립트에 들어 있다.

Table 1. FutonManufacturing 의 스크립트

스크립트	하는 일
`01-schema.sql`	데이터베이스와 테이블 20 개를 생성한다.
`02-sample-data.sql`	품목, 자재 명세, 재고, 구매, 품질 검사, 판매의 데이터를 넣는다.
`03-manufacturing-reports.sql`	생산과 재고에 관한 보고서 뷰 20 개를 생성한다.
`05-sales-schema-enhancements.sql`	판매에 관한 테이블 12 개를 더하고, `Customer` 와 `SalesOrder` 에 컬럼을 더한다.
`06-sales-sample-data.sql`	판매에 더한 테이블의 데이터를 넣는다.
`07-sales-reports.sql`	판매에 관한 보고서 뷰 20 개를 생성한다.

`04` 와 `08` 로 시작하는 스크립트는 예제 쿼리만 담고 있어서, 본서 SQL Server Sample Databases Persistence 장의 SQL Server Sample Databases 의 컨테이너는 이 둘을 실행하지 않는다.

테이블과 뷰

테이블 32 개는 모두 `dbo` 스키마에 있고, 이름은 `BillOfMaterials` 처럼 단어마다 첫 글자를 대문자로 쓴 **PascalCase** 다. 기본 키는 모두 컬럼 하나인 `int IDENTITY` 다. 복합 키도, `uniqueidentifier` 나 `hierarchyid` 같은 SQL Server 의 형도 없다. AdventureWorks 와 WideWorldImporters 에 비하면 가장 평범한 스키마다.

외래 키는 50 개다. 그 가운데 13 개가 `Items` 를 참조한다. 자재 명세와 재고, 구매, 생산, 품질 검사, 판매, 견적, 반품, 가격표가 모두 품목을 참조하므로, `Items` 가 이 데이터베이스의 중심이다. `BillOfMaterials` 는 `Items` 를 두 번 참조한다. 한 번은 만들어지는 품목으로, 한 번은 거기에 들어가는 구성품으로다.

`05-sales-schema-enhancements.sql` 은 원래의 `SalesOrder` 에 판매 경로(`SalesChannel`)와 매장(`Store`), 영업 담당자(`SalesRep`)를 참조하는 외래 키를 더한다. `Customer` 에도 담당 영업 사원과 영업 지역(`SalesTerritory`)을 더한다. 견적(`SalesQuote`)은 주문으로 바뀌면 그 주문을 참조한다. 반품의 품목(`SalesReturnDetail`)은 반품과 주문의 품목, 품목을 함께 참조한다.

테이블을 하는 일에 따라 일곱 묶음으로 나누어, 묶음마다 테이블과 그 사이의 관계를 그림으로 보면 아래와 같다. 재고와 구매, 생산, 판매, 가격표와 판촉의 그림에는 `Items` 를 함께 그렸고, `05-sales-schema-enhancements.sql` 이 더한 테이블은 판매와 견적과 반품, 가격표와 판촉의 그림에 나누어 그렸다. 그림에는 같은 묶음 안의 외래 키만 그렸다. 그림마다 뒤의 표는 앞의 그림에 나오지 않은 테이블이 담는 것과 행 수다.

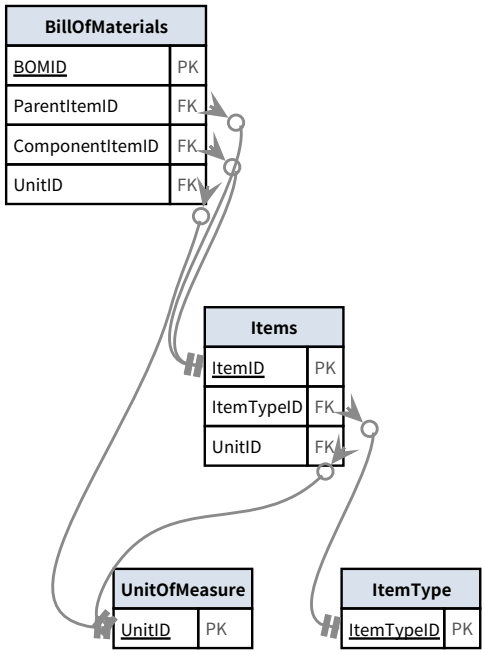


Figure 1. 품목

Table 2. 품목 묶음

테이블	담는 것	행
`UnitOfMeasure`	개, 야드, 파운드 같은 단위	8
`ItemType`	원자재(`RAW`), 구성품(`COMP`), 완제품(`FG`)의 세 품목 종류	3
`Items`	원자재와 구성품, 완제품인 품목과 그 원가, 정가, 재주문 시점, 안전 재고	45
`BillOfMaterials`	상위 품목 하나에 드는 하위 품목과 그 수량, 단위, 손실률, 유효 기간	75

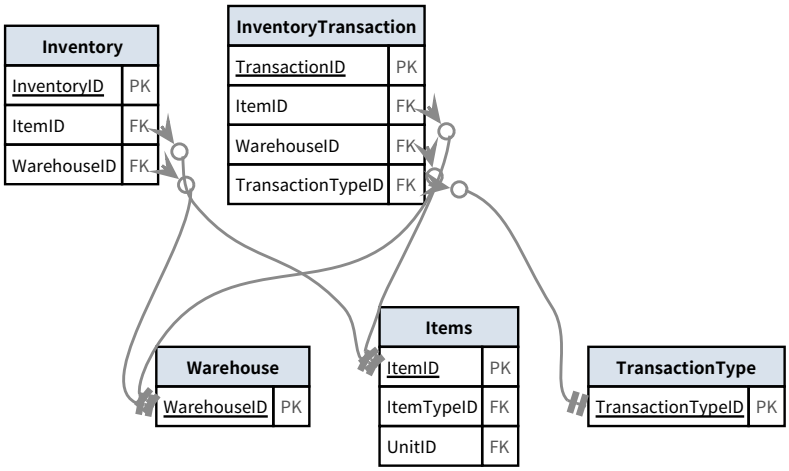


Figure 2. 재고

Table 3. 재고 묶음

테이블	담는 것	행
Warehouse	창고와 그 주소	3
Inventory	창고마다 품목의 재고 수량, 할당된 수량, 가용 수량	45
TransactionType	재고 거래의 종류	9
InventoryTransaction	창고에서 품목이 들고 난 거래	0

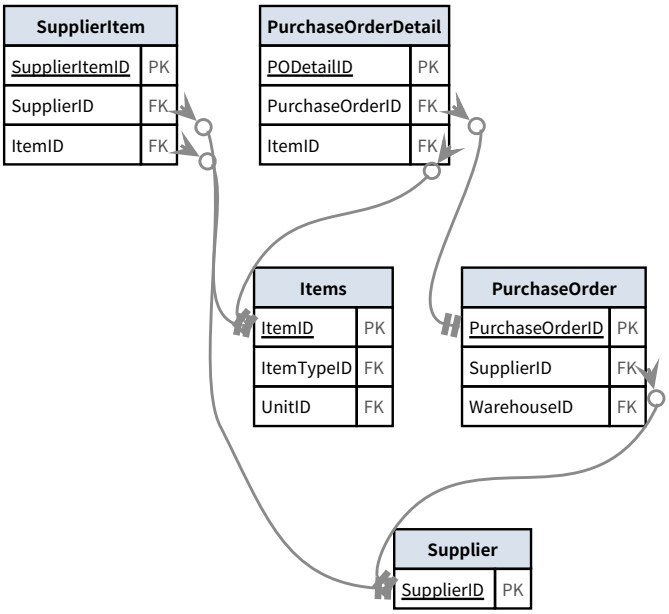


Figure 3. 구매

Table 4. 구매 묶음

테이블	담는 것	행
`Supplier`	자재를 사 오는 공급자와 그 연락처, 결제 조건, 등급	5
`SupplierItem`	공급자마다 공급하는 품목과 그 단가, 최소 주문 수량, 조달 기간	24
`PurchaseOrder`	공급자에게 낸 구매 주문과 그 입고 창고, 금액, 상태	5
`PurchaseOrderDetail`	구매 주문의 품목마다 수량, 단가, 받은 수량	10

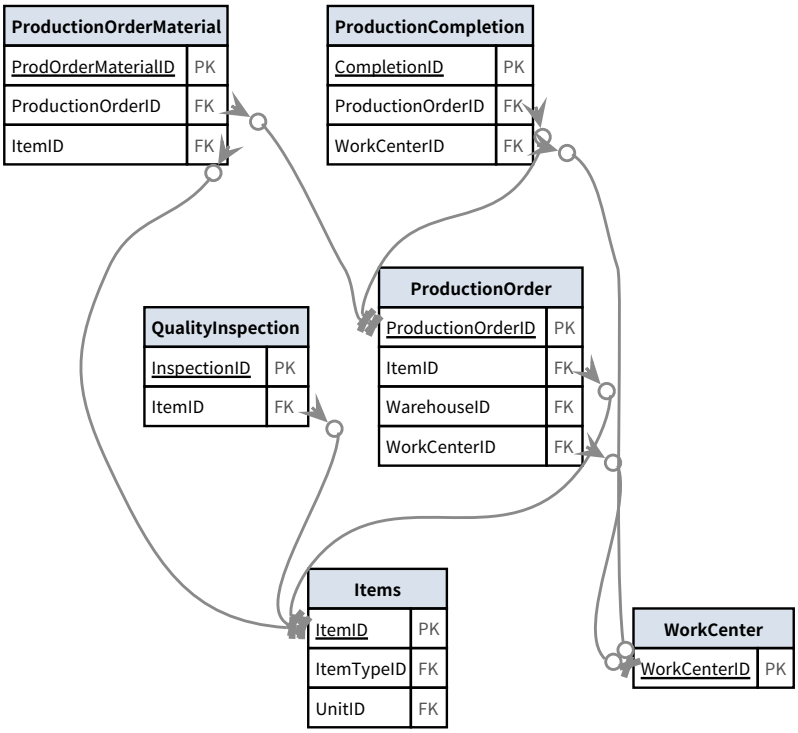


Figure 4. 생산

Table 5. 생산 묶음

테이블	담는 것	행
WorkCenter	작업장과 그 생산 능력	5
ProductionOrder	품목을 만드는 생산 주문과 그 작업장, 수량, 일정, 상태	0
ProductionOrderMaterial	생산 주문에 드는 자재와 그 필요 수량, 출고 수량	0
ProductionCompletion	생산 주문의 완료와 그 양품, 불량 수량	0
QualityInspection	품목의 품질 검사와 그 검사, 합격, 불합격 수량	5

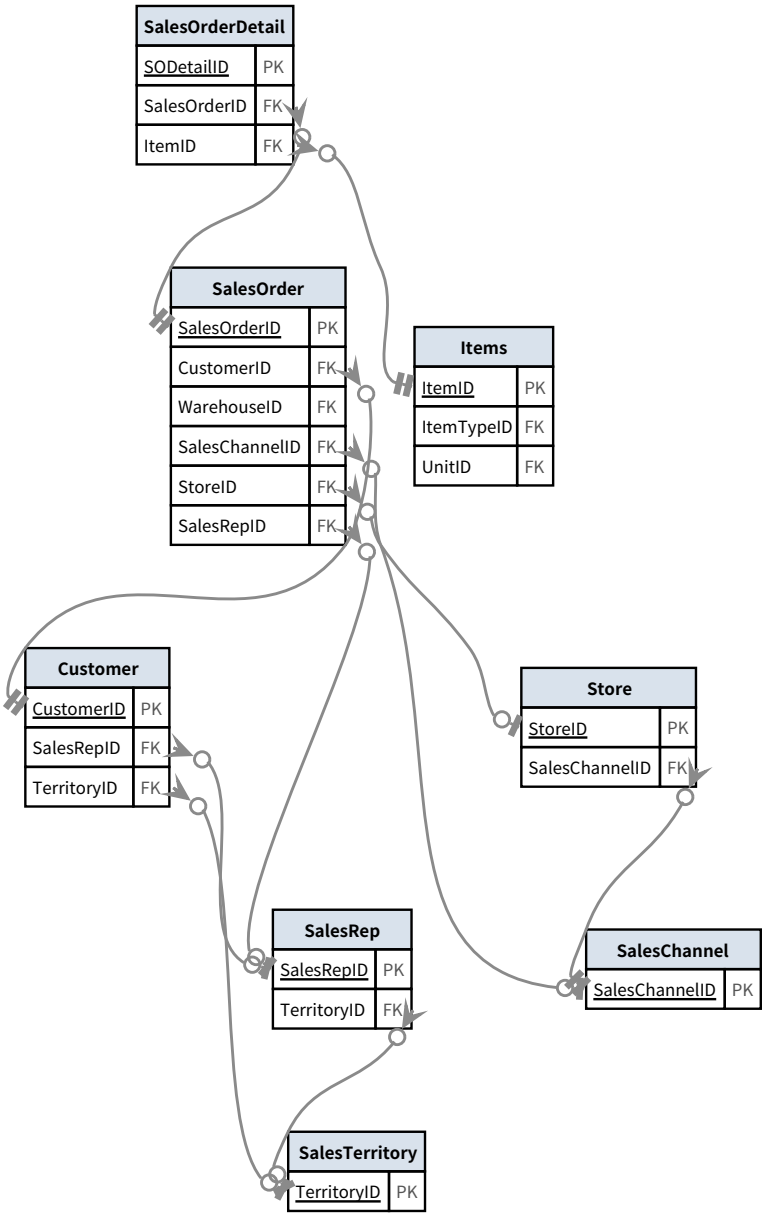

SalesOrderDetail
SODetailID PK
SalesOrderID FK
ItemID FK
SalesOrder
SalesOrderID PK
CustomerID FK
WarehouseID FK
SalesChannelID FK
StoreID FK
SalesRepID FK
Items
ItemID PK
ItemTypeID FK
UnitID FK
Customer
CustomerID PK
SalesRepID FK
TerritoryID FK
Store
StoreID PK
SalesChannelID FK
SalesRep
SalesRepID PK
TerritoryID FK
SalesChannel
SalesChannelID PK
SalesTerritory
TerritoryID PK

Figure 5. 판매

Table 6. 판매 묶음

테이블	담는 것	행
Customer	고객과 그 연락처, 신용 한도, 담당 영업 사원, 영업 지역	8
SalesOrder	고객의 판매 주문과 그 판매 경로, 매장, 영업 담당자, 금액, 상태	15
SalesOrderDetail	판매 주문의 품목마다 수량, 단가, 할인, 출고한 수량	23
SalesChannel	도매, 소매, 온라인의 판매 경로	3
Store	매장과 그 판매 경로, 점장, 주소, 개점일	7
SalesTerritory	영업 지역과 그 권역	6
SalesRep	영업 담당자와 그 사번, 연락처, 영업 지역, 채용일	8

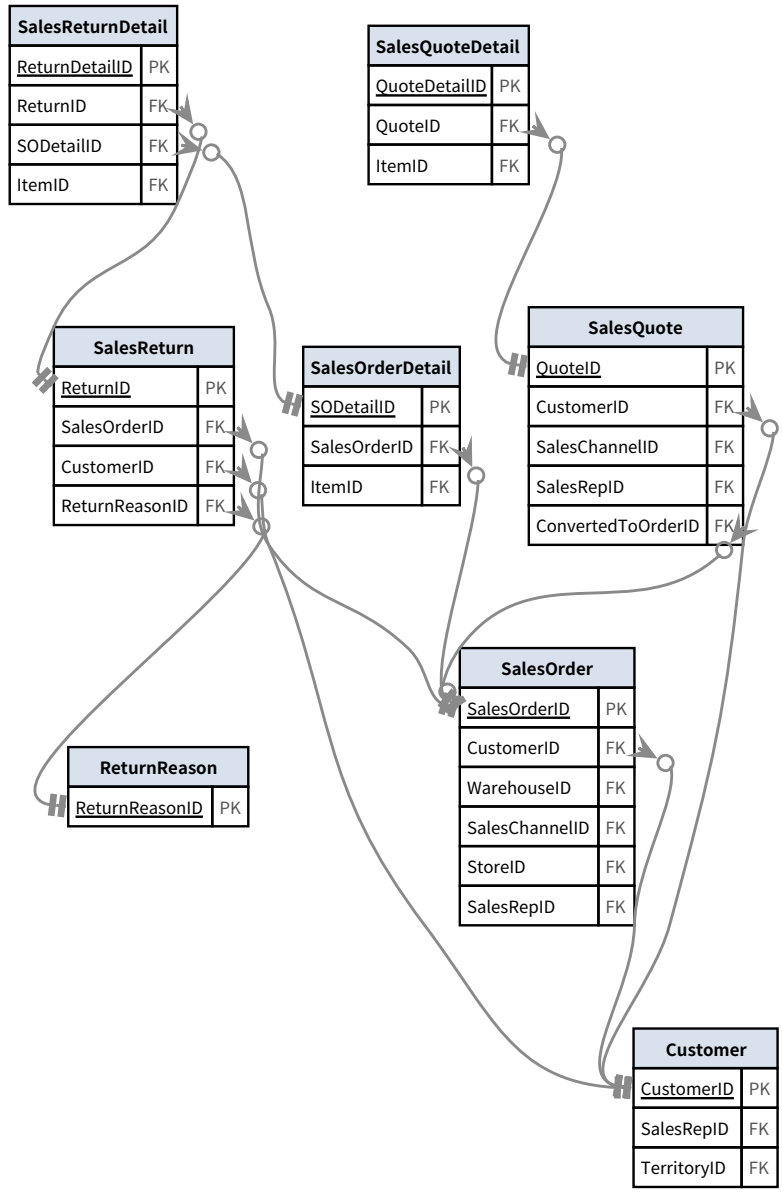


Figure 6. 견적과 반품

Table 7. 견적과 반품 묶음

테이블	담는 것	행
SalesQuote	고객에게 낸 견적과 그 유효 기간, 상태	2
SalesQuoteDetail	견적의 품목마다 수량, 단가, 할인	4
SalesReturn	고객의 반품과 그 주문, 이유, 환불 금액, 재입고 수수료, 상태	2
SalesReturnDetail	반품의 품목마다 주문의 품목, 수량, 환불 금액, 처리 방법	2
ReturnReason	반품의 이유	8

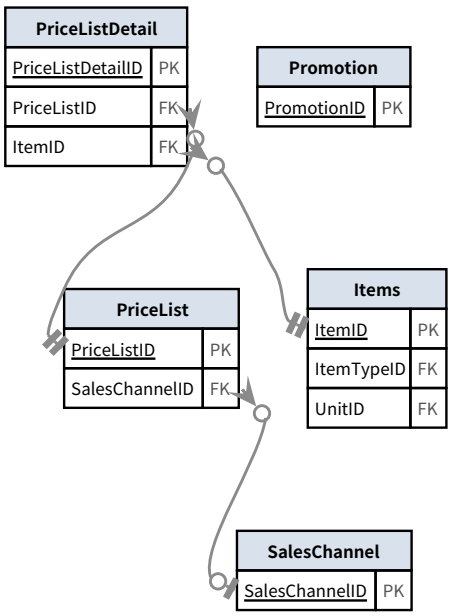


Figure 7. 가격표와 판촉

Table 8. 가격표와 판촉 묶음

테이블	담는 것	행
PriceList	판매 경로마다의 가격표와 그 기간	3
PriceListDetail	가격표마다 품목의 단가와 최소 수량	18
Promotion	판촉과 그 할인, 기간, 최소 구매 금액, 적용할 판매 경로	4

참조가 외래 키로 선언되지 않은 곳도 있다. `QualityInspection` 은 검사한 입고를 `ReferenceType` 이 `PO` 이고 `ReferenceNumber` 가 구매 주문의 번호인 문자열로 기록한다. 데이터베이스가 지키는 관계가 아니므로 엔티티에도 관계가 없다. Exercises 에서 이 두 컬럼으로 구매 주문과 검사를 조인한다.

뷰 40 개는 모두 보고서다. `03-manufacturing-reports.sql` 의 뷰 20 개는 자재 명세의 전개와 역전개, 재고의 평가와 회전율, 생산 주문의 상태, 공급자의 실적 같은 것을 보이고, `07-sales-reports.sql` 의 뷰 20 개는 판매

경로, 매장, 영업 사원, 고객, 상품마다의 판매 실적을 보인다. 판매의 뷰는 이름이 `vw_Sales_` 로 시작한다. 뷰가 어떤 클래스가 되었는지는 보고서 뷰 에서 본다.

행 수

설치한 데이터베이스는 아주 작아서, 가장 큰 `BillOfMaterials` 도 75 행이다. `ProductionOrder`, `ProductionOrderMaterial`, `ProductionCompletion`, `InventoryTransaction` 에는 행이 없다. 예제 데이터에 생산 주문과 재고의 입출고가 들어 있지 않기 때문이다. 그래서 생산 주문을 읽는 뷰도 행을 돌려주지 않는다. 이 장의 연습문제가 생산보다 자재 명세와 재고, 구매, 판매를 묻는 것은 그래서다.

품목 45 개는 원자재(`RAW`) 24 개, 구성품(`COMP`) 15 개, 완제품(`FG`) 6 개다. 원자재는 충전재, 원단, 목재, 금속 부품, 마감재 같은 것으로 공급자에게서 산다. 구성품은 원자재로 만드는 베개, 매트리스, 프레임이고, 완제품은 구성품을 조립한 요다. 자재 명세 75 행 가운데 18 행은 완제품에 구성품을, 57 행은 구성품에 원자재를 연결한다. 그러니 자재 명세의 단계는 완제품, 구성품, 원자재의 셋이다.

몇몇 날짜는 데이터를 넣은 날이다. `BillOfMaterials` 와 `SupplierItem` 의 `EffectiveDate`, 여러 테이블의 `CreatedDate` 는 기본값이 `GETDATE()` 이고, 스크립트가 값을 넣지 않는다. 컨테이너를 새로 시작할 때마다 바뀌는 값이라서, 이 장의 연습문제는 이 날짜를 읽지 않는다.

자재 명세

자재 명세(資材明細, bill of materials)는 품목 하나를 만드는 데 어떤 품목이 얼마나 드는지를 기록한다. `BillOfMaterials` 의 한 행은 부모 품목(`ParentItemID`)에 구성품(`ComponentItemID`)이 `Quantity` 만큼 든다는 뜻이다. `ScrapRate` 는 만들면서 버려지는 양의 백분율이고, `UnitID` 는 수량의 단위다. 두 품목 컬럼은 모두 `Items` 를 참조하고, 검사 제약 조건 `CHK_BOM_NotSelf` 가 품목이 자기 자신의 구성품이 되는 것을 막는다.

구성품도 품목이므로 다시 자기 자재 명세를 가질 수 있다. 완제품 `FG-FUT-003` 은 구성품 셋으로 만들고, 그 가운데 매트리스 `COMP-MAT-003` 은 원자재 둘로 만든다. 이렇게 한 테이블이 여러 단계의 트리를 담는다.

```
FG-FUT-003 Full Deluxe Futon - Microfiber Black
├── COMP-FRM-004 Full Pine Frame - Dark Walnut   1 EA
│   ├── RM-WOOD-001 Pine Frame Rail 6ft          2 EA
│   └── ... 원자재 일곱 개 더
├── COMP-MAT-003 Queen Memory Foam Mattress      1 EA
│   ├── RM-FILL-002 Memory Foam Chips           28 LB
│   └── RM-FAB-004 Microfiber Suede - Black     10 YD
└── COMP-PIL-002 Premium Memory Foam Pillow      2 EA
    ├── RM-FILL-002 Memory Foam Chips            3 LB
    └── RM-FAB-004 Microfiber Suede - Black    1.2 YD
```

같은 원자재가 여러 구성품 아래에 나타난다. `RM-FILL-002` 는 매트리스에도 베개에도 든다. **그러니 이 구조는 엄밀히는 나무가 아니라 방향이 있고 순환이 없는 그래프다.** 원자재 하나가 완제품 하나에 모두 얼마나 드는지 알려면, 여러 경로의 수량을 곱하고 더해야 한다.

두 번 참조하는 테이블

본서 SQL Server Sample Databases Persistence 장의 매핑의 규칙 에서 본 규칙을 따라, 매핑된 슈퍼클래스는 외래 키의 컬럼을 읽기 전용인 기본 속성으로 매핑한다.

MappedBillOfMaterials 의 parentItemId 와 componentItemId

```
// 읽기 전용: 이 컬럼을 쓰는 매핑은 상속하는 엔티티가 갖는다. 클래스의 문서를 보라
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_PARENT_ITEM_ID,
        nullable = false,
        insertable = false,
        updatable = false
)
private Integer parentItemId;

// 읽기 전용: 이 컬럼을 쓰는 매핑은 상속하는 엔티티가 갖는다. 클래스의 문서를 보라
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_COMPONENT_ITEM_ID,
        nullable = false,
        insertable = false,
        updatable = false
)
private Integer componentItemId;
```

두 컬럼을 쓰는 것은 엔티티의 관계다. `BillOfMaterials` 는 `Item` 을 참조하는 `@ManyToOne` 을 둘 갖는다.

BillOfMaterials 의 parentItem 과 componentItem

```
@jakarta.persistence.Entity(name = "DboBillOfMaterials")
@Table(schema = MappedBillOfMaterials.TABLE_SCHEMA,
        name = MappedBillOfMaterials.TABLE_NAME)
public class BillOfMaterials extends MappedBillOfMaterials {
    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(
            name = COLUMN_NAME_PARENT_ITEM_ID,
            referencedColumnName = MappedItem.COLUMN_NAME_ITEM_ID,
            nullable = false,
            insertable = true,
            updatable = true
    )
    private Item parentItem;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = COLUMN_NAME_COMPONENT_ITEM_ID,
                referencedColumnName = MappedItem.COLUMN_NAME_ITEM_ID,
                nullable = false,
                insertable = true,
                updatable = true
    )
    private Item componentItem;
}
```

한 엔티티가 같은 엔티티를 두 번 참조하는 것은 매핑에 아무 문제가 되지 않는다. 두 관계는 `@JoinColumn` 의 `name` 으로 서로 다른 컬럼을 쓴다. `Item` 이 `BillOfMaterials` 를 참조하지 않으므로 자기 참조라는 것은 두 엔티티를 함께 볼 때 드러난다. Jakarta Persistence 에는 다른 엔티티를 참조하는 `@ManyToOne` 둘과 다를 것이 없다.

문제가 되는 것은 이름이다. 관계의 이름을 참조하는 엔티티에서 따오면 두 관계가 모두 `item` 이 된다. **그래서 이 프로젝트는 관계의 이름을 외래 키의 컬럼에서 따온다.** 컬럼의 이름에서 끝의 `ID` 를 떼고 첫 글자를 소문자로 바꾼 것이다. `ParentItemID` 는 `parentItem`, `ComponentItemID` 는 `componentItem`, `UnitID` 는 `unit` 이 된다.

이 규칙은 모듈의 모든 관계에 같게 적용된다. 그래서 관계의 이름이 참조하는 엔티티와 다른 것이 여럿 있다.

Table 9. 컬럼에서 따온 관계의 이름

엔티티	컬럼	관계	참조하는 엔티티
Customer, SalesRep	TerritoryID	territory	SalesTerritory
SalesQuote	ConvertedToOrderID	convertedToOrder	SalesOrder
SalesQuoteDetail	QuoteID	quote	SalesQuote
SalesReturnDetail	SODetailID	soDetail	SalesOrderDetail
SalesReturnDetail	ReturnID	returnSalesReturn	SalesReturn

`ConvertedToOrderID` 에서 따온 `convertedToOrder` 는 엔티티의 이름인 `salesOrder` 보다 관계의 뜻을 잘 드러낸다. 컬럼의 이름은 설계한 사람이 관계에 붙인 이름이기도 하다. 마지막 줄은 예외다. `ReturnID` 에서 따온 `return` 은 자바의 예약어라서 필드의 이름이 될 수 없다. 이때는 참조하는 엔티티의 이름을 덧붙여 `returnSalesReturn` 으로 했다.

반대쪽의 관계

`Item` 에는 관계가 둘 있다. `ItemTypeID` 의 `itemType` 과 `UnitID` 의 `unit` 이다. 자기 자재 명세를 담는 컬렉션은 없다. 본서 SQL Server Sample Databases Persistence 장의 매핑의 규칙 에서 본 것처럼 이 프로젝트의 엔티티는 외래 키마다 `@ManyToOne` 을 하나씩 더할 뿐, `@OneToMany` 는 매핑하지 않는다.

그래서 품목에서 구성품으로 가려면 쿼리가 필요하다. 품목 `item` 의 구성품은 `SELECT b FROM DboBillOfMaterials b WHERE b.parentItem = :item` 으로 읽는다. 응용 프로그램이 원한다면 `Item` 에 `@OneToMany(mappedBy = "parentItem")` 인 컬렉션과 `@OneToMany(mappedBy = "componentItem")` 인 컬렉션을 더할 수 있다. 앞의 것은 구성품을, 뒤의 것은 그 품목을 쓰는 곳을 담는다. 둘 다 `BillOfMaterials` 의 관계가 컬럼을 쓰므로 `mappedBy` 로 그 관계를 참조하기만 하면 된다.

그래도 컬렉션은 한 단계만 담는다. `item.getComponents()` 의 원소에서 다시 `getComponentItem().getComponents()` 를 부르면 다음 단계를 얻지만, 단계마다 지연 로딩이 일어난다. **단계의 수가 정해져 있지 않은 나무를 끝까지 따라가는 일은 관계 매핑만으로 처리하거나 표준 쿼리 하나로 끝낼 수 없다.** 재귀 쿼리 에서 이 문제를 본다.

데이터베이스에만 있는 규칙

`BOMLevel` 은 행이 나무의 몇째 단계인지를 기록한다. 스크립트의 주석은 맨 위가 0 이고 아래로 갈수록 커진다고 설명한다. 설치된 데이터에서는 완제품의 행 18 개가 0 이고, 구성품의 행 57 개가 1 이다. 이 값은 계산된 것이 아니라 저장된 것이다. 구성품을 다른 완제품에도 쓰면 단계가 달라질 수 있는데, 컬럼은 행마다 값이 하나뿐이다. 매핑은 이 컬럼을 그냥 `Integer` 로 읽고, 나무의 구조와 맞는지는 확인하지 않는다.

`CHK_BOM_NotSelf` 도 매핑에는 없다. Jakarta Persistence 3.2 가 더한 검사 제약 조건은 스키마를 생성할 때 쓰는 것이고, 이 프로젝트는 스키마를 생성하지 않는다. 부모와 구성품이 같은 행을 저장하면 데이터베이스가 거부한다. 이 제약 조건이 막는 것은 한 단계의 순환뿐이다. `A` 의 구성품이 `B` 이고 `B` 의 구성품이 `A` 인 두 행은 막지 못한다. 자재 명세를 끝까지 따라가는 코드는 순환이 없다고 가정한다.

재귀 쿼리

자재 명세를 끝까지 펼치는 일을 **전개**(展開, explosion)라고 한다. 완제품에서 시작해 구성품으로, 구성품에서 원자재로 내려가며 단계마다 수량을 곱한다. 반대로 원자재에서 시작해 그것을 쓰는 품목으로 올라가는 일은 역전개(逆展開, where-used)다. FutonManufacturing 은 이 두 가지를 뷰 `vw_BOMExplosion` 과 `vw_WhereUsed` 로 제공한다.

vw_BOMExplosion

`vw_BOMExplosion` 은 재귀 공통 테이블 식(recursive common table expression)으로 정의되어 있다.[2]

vw_BOMExplosion

```
CREATE OR ALTER VIEW vw_BOMExplosion AS
WITH BOMRecursive AS (
    -- 앵커: 맨 위 단계
    SELECT
        b.ParentItemID,
        p.ItemCode AS ParentItemCode,
        p.ItemName AS ParentItemName,
        b.ComponentItemID,
        c.ItemCode AS ComponentItemCode,
        c.ItemName AS ComponentItemName,
        c.ItemTypeID,
        t.TypeName AS ComponentType,
        CAST(b.Quantity AS DECIMAL(18,4)) AS Quantity,
        b.UnitID,
        u.UnitCode,
        b.ScrapRate,
        b.BOMLevel,
        CAST(b.Quantity * (1 + b.ScrapRate/100) AS DECIMAL(18,4)) AS EffectiveQuantity,
        c.StandardCost,
        CAST(b.Quantity * (1 + b.ScrapRate/100) * c.StandardCost AS DECIMAL(18,4))
            AS ExtendedCost,
        1 AS Level,
        CAST(p.ItemCode + ' > ' + c.ItemCode AS NVARCHAR(MAX)) AS BOMPath
    FROM BillOfMaterials b
    INNER JOIN Items p ON b.ParentItemID = p.ItemID
    INNER JOIN Items c ON b.ComponentItemID = c.ItemID
    INNER JOIN ItemType t ON c.ItemTypeID = t.ItemTypeID
    INNER JOIN UnitOfMeasure u ON b.UnitID = u.UnitID
    WHERE b.IsActive = 1

    UNION ALL

    -- 재귀: 하위 구성품을 얻는다
    SELECT
        br.ParentItemID,
```

```
        br.ParentItemCode,
        br.ParentItemName,
        b.ComponentItemID,
        c.ItemCode,
        c.ItemName,
        c.ItemTypeID,
        t.TypeName,
        CAST(br.EffectiveQuantity * b.Quantity AS DECIMAL(18,4)) AS Quantity,
        b.UnitID,
        u.UnitCode,
        b.ScrapRate,
        b.BOMLevel,
        CAST(br.EffectiveQuantity * b.Quantity * (1 + b.ScrapRate/100) AS DECIMAL(18,4))
            AS EffectiveQuantity,
        c.StandardCost,
        CAST(br.EffectiveQuantity * b.Quantity * (1 + b.ScrapRate/100) * c.StandardCost
             AS DECIMAL(18,4)) AS ExtendedCost,
        br.Level + 1,
        CAST(br.BOMPath + ' > ' + c.ItemCode AS NVARCHAR(MAX))
    FROM BOMRecursive br
    INNER JOIN BillOfMaterials b ON br.ComponentItemID = b.ParentItemID
    INNER JOIN Items c ON b.ComponentItemID = c.ItemID
    INNER JOIN ItemType t ON c.ItemTypeID = t.ItemTypeID
    INNER JOIN UnitOfMeasure u ON b.UnitID = u.UnitID
    WHERE b.IsActive = 1
)
SELECT ParentItemID, ParentItemCode, ParentItemName, ComponentItemID, ComponentItemCode,
       ComponentItemName, ComponentType, Level, EffectiveQuantity, UnitCode, StandardCost,
       ExtendedCost, BOMPath
FROM BOMRecursive;
GO
```

`WITH` 로 정의한 `BOMRecursive` 는 `UNION ALL` 로 이은 두 `SELECT` 다. 앞의 것은 앵커 멤버로, 활성인 자재 명세의 행을 모두 읽는다. 뒤의 것은 재귀 멤버로, `BOMRecursive` 자신을 읽는다. 지금까지 얻은 행의 구성품(`br.ComponentItemID`)을 부모로 하는 자재 명세의 행을 찾아, 수량을 곱하고 `Level` 을 하나 늘리고 `BOMPath` 를 잇는다. SQL Server 는 재귀 멤버가 더 이상 행을 돌려주지 않을 때까지 이것을 되풀이한다. SQL Server 의 `WITH` 에는 `RECURSIVE` 라는 키워드가 없다. **자기 자신을 읽는 공통 테이블 식이 재귀다.**

앵커 멤버에 시작 품목을 제한하는 조건이 없으므로, 완제품뿐 아니라 구성품도 전개의 시작이 된다. 그래서 뷰의 행은 자재 명세의 75 행과 그 아래 단계의 69 행을 더한 144 행이다. 완제품 하나의 전개는 `ParentItemCode` 로 선택한다.

```
SELECT Level, EffectiveQuantity, ExtendedCost, BOMPath
FROM vw_BOMExplosion
WHERE ParentItemCode = 'FG-FUT-003'
ORDER BY BOMPath;
```

결과

Level	EffectiveQuantity	ExtendedCost	BOMPath
1	1.0050	217.4117	FG-FUT-003 > COMP-FRM-004
2	.8291	14.9243	FG-FUT-003 > COMP-FRM-004 > RM-FIN-001
2	.8291	18.2408	FG-FUT-003 > COMP-FRM-004 > RM-FIN-003
2	1.0050	8.0400	FG-FUT-003 > COMP-FRM-004 > RM-HARD-001
2	8.1204	22.3311	FG-FUT-003 > COMP-FRM-004 > RM-METAL-001
2	2.0301	31.4666	FG-FUT-003 > COMP-FRM-004 > RM-METAL-002
2	2.1105	25.3260	FG-FUT-003 > COMP-FRM-004 > RM-WOOD-001
2	2.1105	17.9393	FG-FUT-003 > COMP-FRM-004 > RM-WOOD-002
2	10.5525	79.1438	FG-FUT-003 > COMP-FRM-004 > RM-WOOD-003
1	1.0050	380.2418	FG-FUT-003 > COMP-MAT-003
2	10.5525	126.6300	FG-FUT-003 > COMP-MAT-003 > RM-FAB-004
2	28.9842	253.6118	FG-FUT-003 > COMP-MAT-003 > RM-FILL-002
1	2.0200	84.6279	FG-FUT-003 > COMP-PIL-002
2	2.5452	30.5424	FG-FUT-003 > COMP-PIL-002 > RM-FAB-004
2	6.1812	54.0855	FG-FUT-003 > COMP-PIL-002 > RM-FILL-002

RM-FAB-004 와 RM-FILL-002 가 두 번씩 나온다. 매트리스를 거치는 경로와 베개를 거치는 경로가 다르기 때문이다. 그래서 이 뷰에는 식별자로 쓸 컬럼이 없다. ParentItemID 와 ComponentItemID 를 합쳐도 행마다 다르지 않고, 행마다 다른 것은 경로를 이은 문자열 BOMPath 뿐이다. 본서 SQL Server Sample Databases Persistence 장의 행의 식별자 에서 본 것처럼 식별자로 보이지 않는 컬럼은 식별자로 쓰지 않았다. vw_BOMExplosion 은 엔티티가 아닌 클래스 BomExplosion 가 되었다.

BomExplosion 의 생성자

```
public class BomExplosion extends MappedBomExplosion {
    public BomExplosion(final Integer parentItemId,
                        final String parentItemCode,
                        final String parentItemName,
                        final Integer componentItemId,
                        final String componentItemCode,
                        final String componentItemName,
                        final String componentType,
                        final Integer level,
                        final BigDecimal effectiveQuantity,
                        final String unitCode,
                        final BigDecimal standardCost,
                        final BigDecimal extendedCost,
                        final String bomPath) {
        super();
        setParentItemId(parentItemId);
        setParentItemCode(parentItemCode);
        setParentItemName(parentItemName);
        setComponentItemId(componentItemId);
        setComponentItemCode(componentItemCode);
        setComponentItemName(componentItemName);
        setComponentType(componentType);
        setLevel(level);
        setEffectiveQuantity(effectiveQuantity);
        setUnitCode(unitCode);
        setStandardCost(standardCost);
        setExtendedCost(extendedCost);
        setBomPath(bomPath);
    }
}
```

이 클래스는 관리되는 형(型, type)이 아니므로 Query Language 로 읽을 수 없다. 네이티브 쿼리로 뷰의 행을 읽고, 행마다 이 생성자를 불러 객체를 생성한다. vw_WhereUsed 를 이렇게 읽는 예는 Exercises 에 있다. @SqlResultSetMapping 의 @ConstructorResult 로 구현체가 생성자를 부르게 할 수도 있지만, 그 애너테이션은 엔티티나 매핑된 슈퍼클래스에 두어야 한다. 이 프로젝트는 매핑된 슈퍼클래스에 데이터베이스가 정한 것만 두므로, 쿼리의 결과를 정하는 매핑은 두지 않았다.

Jakarta Persistence 에는 재귀 쿼리가 없다

Query Language 에는 `WITH` 가 없다. Criteria API 에도 공통 테이블 식을 생성하는 메서드가 없다. 온라인 부록 Oracle HR Persistence 에서 직원의 상사를 따라갈 때처럼, 단계의 수가 정해지지 않은 계층을 한 쿼리로 따라갈 수는 없다. Hibernate ORM 의 HQL 은 `WITH RECURSIVE` 를 지원하지만, 구현체의 확장이다. 이 프로젝트의 구현체는 Hibernate ORM 하나뿐이지만, 연습문제는 Jakarta Persistence 의 쿼리로만 풀었으므로 이 확장을 쓰지 않았다.

그러면 남는 방법은 둘이다. 하나는 재귀 공통 테이블 식을 네이티브 쿼리로 보내는 것이다. 뷰가 있으면 뷰를 읽으면 되고, 없으면 SQL 을 직접 작성한다. 쿼리 하나로 끝나고, 계산도 데이터베이스가 한다. 결과는 엔티티가 아니라 값의 배열이나 엔티티가 아닌 클래스다.

다른 하나는 자바로 나무를 따라가는 것이다. 한 단계의 행을 쿼리로 읽고, 그 구성품을 부모로 하는 다음 단계의 행을 다시 읽는다. 구성품이 없는 품목이 나오면 멈춘다. 쿼리는 단계마다 하나씩 나가고, 결과는 엔티티다. 매핑된 슈퍼클래스의 읽기 전용 속성 `parentItemId` 가 여기서 쓸모가 있다. `b.parentItemId IN :parentItemIds` 로 한 단계의 부모를 모두 넘기면, 관계를 로드하지 않고도 단계의 행을 한 번에 읽는다.

FutonManufacturing 의 나무는 세 단계뿐이라 자바로 따라가도 쿼리는 셋이다. **단계가 깊고 행이 많으면 재귀 쿼리가 낫다.** 어느 쪽이든 순환이 있으면 끝나지 않는다. SQL Server 는 재귀가 100 번을 넘으면 쿼리를 멈추고 오류를 낸다. 자바의 반복문에는 그런 장치가 없으므로, 순환이 있을 수 있다면 방문한 품목을 기억해야 한다.

뷰의 계산을 그대로 읽는다

뷰를 엔티티로 매핑하면 뷰가 계산한 값을 그대로 읽는다. 매핑은 계산이 맞는지 알지 못한다. `vw_BOMExplosion` 을 읽는 `vw_CostRollUp` 이 그런 예다. 이 뷰는 `ItemCode` 로 식별되는 엔티티 `CostRollUp` 가 되었다.

vw_CostRollUp 의 ItemCosts

```
CREATE OR ALTER VIEW vw_CostRollUp AS
WITH ItemCosts AS (
    SELECT
        ParentItemID,
        ParentItemCode,
        ParentItemName,
        SUM(ExtendedCost) AS TotalMaterialCost,
        COUNT(DISTINCT ComponentItemID) AS NumberOfComponents
    FROM vw_BOMExplosion
    GROUP BY ParentItemID, ParentItemCode, ParentItemName
)
```

`ItemCosts` 는 부모 품목마다 전개의 `ExtendedCost` 를 모두 더한다. 그런데 전개에는 단계마다 행이 있다. 1 단계의 구성품 행의 `ExtendedCost` 는 구성품의 표준 원가로 계산한 것이고, 구성품의 표준 원가는 이미 그 원자재의 원가를 더한 값이다. 여기에 2 단계의 원자재 행을 또 더하므로 재료비가 두 번 더해진다.

```
SELECT ItemCode, CurrentStandardCost, CalculatedMaterialCost, GrossMarginPercent
FROM vw_CostRollUp
WHERE ItemType = 'Finished Goods'
ORDER BY ItemCode;
```

결과

```
ItemCode    CurrentStandardCost  CalculatedMaterialCost  GrossMarginPercent
FG-FUT-001  .0000                628.6937                100.00
FG-FUT-002  .0000                847.6106                100.00
FG-FUT-003  .0000                1364.5630               100.00
FG-FUT-004  .0000                1429.0842               100.00
FG-FUT-005  .0000                988.6890                100.00
FG-FUT-006  .0000                1449.1838               100.00
```

FG-FUT-003 의 `CalculatedMaterialCost` 1364.5630 은 앞의 전개에서 1 단계 행의 `ExtendedCost` 합 682.2814 와 2 단계 행의 합 682.2816 을 더한 것이다. 두 합이 마지막 자리에서 다른 것은 단계마다 소수(小數, decimal fraction) 넷째 자리로 반올림했기 때문이다. 실제 재료비는 그 절반쯤이다. 완제품의 `StandardCost` 가 모두 0 이라서 `GrossMarginPercent` 도 모두 100 이다.

`CostRollUp` 는 이 값을 충실하게 읽을 뿐이다. **뷰를 매핑하는 것은 뷰의 정의까지 받아들이는 일이다.** 완제품마다 올바른 재료비와 이익은 Exercises 에서 직접 구한다.

보고서 뷰

FutonManufacturing 의 뷰 40 개 가운데 18 개는 엔티티가, 22 개는 엔티티가 아닌 클래스가 되었다. 기준은 본서 SQL Server Sample Databases Persistence 장의 행의 식별자 에서 본 것과 같다. 뷰가 돌려주는 모든 행에서 값이 서로 다르고 `NULL` 이 아니며, 이름이 …`ID`, …`Code`, …`Number` 처럼 식별자로 보이는 컬럼이 있으면 그 컬럼으로 식별한다.

엔티티가 된 뷰

엔티티가 된 뷰는 대부분 품목, 고객, 공급자, 주문 같은 것 하나에 한 행을 돌려주는 보고서다. 식별자는 그 대상의 코드나 번호다.

Table 10. 엔티티가 된 뷰

뷰	엔티티	행 수	식별자
vw_CostRollUp	CostRollUp	45	ItemCode
vw_InventoryTurnover	InventoryTurnover	45	ItemCode
vw_InventoryValuation	InventoryValuation	45	ItemCode
vw_Sales_ProductPerformance	SalesProductPerformance	6	ItemCode
vw_CustomerFulfillmentRate	CustomerFulfillmentRate	8	CustomerCode
vw_Sales_CustomerAnalysis	SalesCustomerAnalysis	8	CustomerID
vw_Sales_CustomerLifetimeValue	SalesCustomerLifetimeValue	8	CustomerID
vw_Sales_CustomerSegmentation	SalesCustomerSegmentation	7	CustomerID
vw_SupplierPerformance	SupplierPerformance	5	SupplierCode
vw_PurchaseOrderStatus	PurchaseOrderStatus	5	PONumber
vw_SalesOrderBacklog	SalesOrderBacklog	6	OrderNumber
vw_Sales_QuoteConversion	SalesQuoteConversion	2	QuoteNumber
vw_Sales_ReturnsAnalysis	SalesReturnsAnalysis	2	ReturnID
vw_WorkCenterCapacity	WorkCenterCapacity	5	WorkCenterCode
vw_Sales_ByTerritory	SalesByTerritory	6	TerritoryCode
vw_Sales_ChannelProfitability	SalesChannelProfitability	3	ChannelCode
vw_Sales_RepPerformance	SalesRepPerformance	8	EmployeeCode
vw_Sales_StorePerformance	SalesStorePerformance	7	StoreCode

뷰를 매핑한 엔티티에도 테이블을 매핑한 엔티티와의 관계는 없다. `vw_Sales_CustomerAnalysis` 의 `CustomerID` 는 `Customer` 의 기본 키와 값이 같지만, 뷰에는 외래 키가 없고 엔티티에도 관계가 없다. **뷰의 엔티티는 읽기만 하는 독립된 엔티티다.** 고객의 분석을 고객과 함께 보려면 Query Language 에서 `JOIN DboSalesCustomerAnalysis v ON v.customerId = c.customerId` 처럼 엔티티를 조인한다.

식별자는 매핑된 슈퍼클래스에 `@Id` 로 기재된다. 뷰의 컬럼은 식별자까지 모두 `insertable = false, updatable = false` 로 읽기만 한다. SQL Server 가 이 뷰들의 행 전체를 `INSERT` 나 `UPDATE` 로 받지 않기 때문이다. 행이 여러 기본 테이블에 걸쳐 있거나, 계산한 컬럼이나 집계한 컬럼, 상수인 컬럼이 있다. 식별자를 정한 이유와 읽기만 하는 이유는 클래스의 문서가 밝힌다.

MappedInventoryValuation 의 itemCode

```
/**
 * {@code dbo.vw_InventoryValuation} 뷰의 매핑을 담는 매핑된 슈퍼클래스.
 * <p>
 * 식별자: 뷰는 키를 선언하지 않는다. ItemCode 는 뷰가 돌려주는 모든 행에서 값이 서로 다르고 널이
 * 아니다.
 * <p>
 * 읽기 전용: SQL Server 는 이 뷰의 행 전체를 INSERT 나 UPDATE 로 받지 않으므로, 어느 컬럼도
 * 갱신하거나 삽입할 수 없다. 행 전체를 INSERT 하거나 UPDATE 하면 기본 테이블 둘 이상이 바뀌고
 * (오류 4405), 몇몇 컬럼은 파생되었거나 상수이며 (오류 4406), 한 컬럼은 기본 테이블의 계산되는
 * 컬럼이다 (오류 271).
 */
@MappedSuperclass
public abstract class MappedInventoryValuation {
    // 읽기 전용: SQL Server 는 이 뷰에 쓰지 않는다. 클래스의 문서를 보라
    @Size(max = COLUMN_LENGTH_ITEM_CODE)
    @NotNull
    @Id
    @Column(name = COLUMN_NAME_ITEM_CODE,
            nullable = false,
            insertable = false,
            updatable = false,
            length = COLUMN_LENGTH_ITEM_CODE
    )
    private String itemCode;
```

데이터가 정한 식별자

`vw_InventoryValuation` 의 식별자 `ItemCode` 는 지금의 데이터에서만 행마다 다르다.

이 뷰는 `Inventory` 의 행마다 한 행을 돌려주고, `Inventory` 의 행은 품목과 창고마다 하나다. 창고는 셋이지만, 설치된 재고 45 행은 모두 본사 공장 `WH-MAIN` 에 있다. 그래서 품목의 코드가 행마다 다를 뿐이다. 다른 창고에 같은 품목의 재고가 생기면 `ItemCode` 가 같은 행이 둘이 된다.

그때 이 엔티티를 읽으면 잘못된 결과를 얻는다. 영속성 컨텍스트에서 엔티티는 식별자마다 하나뿐이다. 구현체는 같은 식별자의 두 번째 행을 읽을 때 이미 있는 엔티티를 돌려준다. 그래서 쿼리의 결과에는 같은 객체가 두 번 들어 있고, 두 번째 창고의 수량은 보이지 않는다. 오류도 나지 않는다.

이런 뷰에 맞는 식별자는 `WarehouseCode` 와 `ItemCode` 를 합친 복합 키다. 이 프로젝트가 컬럼 하나를 선택한 것은 데이터에서 확인할 수 있는 것만 식별자로 삼았기 때문이다. **뷰를 엔티티로 매핑할 때는 지금의 데이터가 아니라**

뷰의 정의를 보고 식별자를 정해야 한다. 정의에서 행이 무엇마다 하나인지를 읽어 내면 된다. `vw_InventoryTurnover` 도 `Items` 에 `Inventory` 를 조인하므로 같은 문제가 있다. `vw_CostRollUp` 은 `Items` 의 행마다 한 행을 돌려주므로 `ItemCode` 가 정의에서도 식별자다.

엔티티가 아닌 뷰

엔티티가 되지 못한 뷰 22 개는 이유에 따라 셋으로 나눌 수 있다.

Table 11. 엔티티가 아닌 클래스가 된 뷰

이유	뷰
행이 없다	`vw_ComponentShortage`, `vw_DailyProductionSchedule`, `vw_ItemsBelowReorderPoint`, `vw_LateProductionOrders`, `vw_MaterialRequirements`, `vw_ProductionCompletionSummary`, `vw_ProductionOrderStatus`, `vw_RawMaterialUsage`, `vw_ScrapWasteAnalysis`
같은 품목이 여러 경로로 나온다	`vw_BOMExplosion`, `vw_WhereUsed`
기간이나 구간, 순위로 묶는다	`vw_QualityInspectionSummary`, `vw_Sales_ByChannel`, `vw_Sales_DiscountAnalysis`, `vw_Sales_GrowthAnalysis`, `vw_Sales_OrderSizeDistribution`, `vw_Sales_OrderValueAnalysis`, `vw_Sales_Pipeline`, `vw_Sales_ProductMixAnalysis`, `vw_Sales_TimeBasedAnalysis`, `vw_Sales_TopProducts`, `vw_Sales_TrendAnalysis`

행이 없는 뷰 아홉 개 가운데 여덟 개는 행이 없는 생산 주문과 재고의 입출고를 읽는다. `vw_ItemsBelowReorderPoint` 는 재주문점 아래로 떨어진 품목이 지금 없을 뿐이다. 행이 없으면 어느 컬럼이 행마다 다른지 확인할 수 없으므로 엔티티가 아닌 클래스가 되었다. 데이터가 생기면 판단이 바뀔 수 있다.

기간으로 묶는 뷰는 `Year` 와 `Month`, 판매 경로나 고객 유형 같은 여러 컬럼이 합쳐져야 행이 정해진다. `vw_Sales_ByChannel` 은 판매 경로와 달마다 한 행이므로 `ChannelCode` 가 달마다 되풀이된다. `vw_Sales_Pipeline` 의 `Stage` 는 행마다 다르지만, 판매 단계의 이름이지 식별자로 보이는 컬럼은 아니다. 이런 뷰도 뷰의 정의를 보고 복합 키를 정하면 엔티티로 매핑할 수 있다. 이 프로젝트는 그 판단을 응용 프로그램에 맡기고, 컬럼을 모두 받는 생성자만 둔다.

Attribute Mappings

FutonManufacturing 의 테이블 32 개와 뷰 40 개에서 매핑된 슈퍼클래스가 컬럼을 매핑한 속성은 모두 878 개이고, 컬럼마다 속성이 꼭 하나씩 있다. SQL Server 의 형을 어떤 자바의 형으로 매핑하는지는 본서 SQL Server Sample Databases Persistence 장의 SQL Server 의 형 에서 보았다. 이 절은 먼저 FutonManufacturing 의 컬럼에 그 표를 어떻게 적용했는지 요약하고, 그다음 FutonManufacturing 의 소수와 이름을 살펴본다. 테이블을 하나씩 살펴보지는 않는다.

요약

- `bigint` 인 `Long` 은 뷰가 계산한 순위와 개수다.
- `decimal` 과 `numeric` 은 `BigDecimal` 이다(소수와 이름). `numeric` 은 뷰가 계산한 값이다.
- 하나뿐인 `float` 는 `Double` 이다. 뷰 `vw_Sales_OrderValueAnalysis` 가 `STDEV` 로 계산한 표준 편차다.
- `datetime` 과 `varchar` 는 뷰의 컬럼에만 있다.

소수와 이름

FutonManufacturing 의 수량과 금액은 모두 `decimal` 이다. 대부분 `decimal(18, 2)` 와 `decimal(18, 4)` 이고, 백분율은 `decimal(5, 2)`, 공급자의 평점은 `decimal(3, 2)` 다. `money` 는 쓰지 않는다. 본서 SQL Server Sample Databases Persistence 장의 SQL Server 의 형 에서 본 것처럼 `decimal` 은 `BigDecimal` 이 되고, `@Column` 의 `precision` 과 `scale`, 그리고 `@Digits` 를 갖는다.

precision, scale 과 @Digits

자재 명세의 수량은 소수(小數, decimal fraction) 넷째 자리까지, 손실률은 둘째 자리까지 기록한다.

MappedBillOfMaterials 의 quantity 와 scrapRate

```
@jakarta.validation.constraints.Digits(
        integer = COLUMN_PRECISION_QUANTITY - COLUMN_SCALE_QUANTITY,
        fraction = COLUMN_SCALE_QUANTITY)
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_QUANTITY,
        nullable = false,
        insertable = true,
        updatable = true,
        precision = COLUMN_PRECISION_QUANTITY,
        scale = COLUMN_SCALE_QUANTITY
)
private BigDecimal quantity;

@Digits(integer = COLUMN_PRECISION_SCRAP_RATE - COLUMN_SCALE_SCRAP_RATE,
        fraction = COLUMN_SCALE_SCRAP_RATE)
@Basic(optional = true)
@Column(name = COLUMN_NAME_SCRAP_RATE,
        nullable = true,
        insertable = true,
        updatable = true,
        precision = COLUMN_PRECISION_SCRAP_RATE,
        scale = COLUMN_SCALE_SCRAP_RATE
)
private BigDecimal scrapRate;
```

`precision` 과 `scale` 은 스키마를 생성할 때 컬럼의 형을 정하는 데 쓰인다. 이 프로젝트는 스키마를 생성하지 않으므로, 이 둘은 데이터베이스의 형을 기록해 둔 것에 가깝다. 값을 확인하는 것은 Jakarta Validation 의 `@Digits` 다. `integer` 는 정수부의 자릿수, `fraction` 은 소수부의 자릿수이므로, `precision` 에서 `scale` 을 뺀 것과 `scale` 을 넘긴다. `Quantity` 는 정수부 14 자리, 소수부 4 자리까지 허용한다.

두 확인은 동작이 다르다. SQL Server 는 소수부의 자릿수가 많은 값을 받으면 `scale` 에 맞추어 반올림해 저장하고 오류를 내지 않는다. `1.23456` 을 `Quantity` 에 저장하면 `1.2346` 이 된다. `@Digits` 는 이 값을 받아들이지 않는다. 영속성 유닛의 `validation-mode` 가 `CALLBACK` 이므로, 구현체는 엔티티를 저장하기 전에 검증하고 위반이 있으면 `ConstraintViolationException` 을 던진다. 조용히 반올림되는 것보다 저장하지 못하는 것이 낫다면 `@Digits` 가 그 일을 한다. 정수부가 넘치는 값은 데이터베이스도 거부한다.

`BigDecimal` 은 비교할 때도 주의해야 한다. `equals` 는 값과 함께 `scale` 도 비교하므로 `new BigDecimal("1.0")` 과 `new BigDecimal("1.0000")` 은 같지 않다. 같은 컬럼을 읽으면 `scale` 이 같지만, 쿼리가 계산한 값의 `scale` 은 데이터베이스가 정한다. Exercises 의 재료비 `SUM(b.quantity * (1 + b.scrapRate / 100) * c.standardCost)` 는 SQL Server 에서 소수부가 여섯 자리인 값이 된다. 그래서 연습문제의 테스트는 값을 `compareTo` 로 비교하거나 같은 `scale` 로 반올림한 다음 비교한다.

계산되는 컬럼

계산되는 컬럼은 일곱 개다. `Inventory` 의 `QuantityAvailable` 은 `QuantityOnHand` 에서 `QuantityAllocated` 를 뺀 것이고, 주문과 견적의 품목의 `LineTotal` 은 수량과 단가를 곱한 것이다. `SalesOrderDetail` 에는 할인 금액과 순 금액도 계산된다.

MappedSalesOrderDetail 의 lineTotal

```
/**
 * {@code decimal(37, 6), computed as ([Quantity]*[UnitPrice])} 컬럼의 이름. 값은
 * {@value} 이다.
 */
public static final String COLUMN_NAME_LINE_TOTAL = "LineTotal";

// 읽기 전용: 계산되는 컬럼이다. SQL Server 가 값을 계산하고, 값을 넣으면 거부한다 (오류 271)
@Digits(integer = COLUMN_PRECISION_LINE_TOTAL - COLUMN_SCALE_LINE_TOTAL,
        fraction = COLUMN_SCALE_LINE_TOTAL)
@Basic(optional = true)
@Column(name = COLUMN_NAME_LINE_TOTAL,
        nullable = true,
        insertable = false,
        updatable = false,
        precision = COLUMN_PRECISION_LINE_TOTAL,
        scale = COLUMN_SCALE_LINE_TOTAL
)
private BigDecimal lineTotal;
```

계산되는 컬럼은 `insertable = false, updatable = false` 로 읽기만 한다. 형은 SQL Server 가 식에서 정한 `decimal(37, 6)` 이다. `decimal(18, 2)` 인 수량과 `decimal(18, 4)` 인 단가를 곱하면, 정밀도는 두 정밀도의 합에 1 을 더한 37 이 되고 소수부의 자릿수는 두 자릿수의 합인 6 이 된다.

읽기 전용인 속성은 저장한 뒤에도 값이 바뀌지 않는다. 새 `SalesOrderDetail` 을 `persist` 하고 플러시해도 `getLineTotal()` 은 `null` 이다. 구현체가 `INSERT` 에 넣지 않은 컬럼을 다시 읽지 않기 때문이다. 데이터베이스가 계산한 값이 필요하면 `refresh` 로 엔티티를 다시 읽는다. Hibernate ORM 의 `@Generated` 처럼 저장한 뒤에 값을 다시 읽게 하는 애너테이션은 구현체의 확장이고, Jakarta Persistence 에는 없다.

이름

테이블과 컬럼의 이름은 PascalCase 다. 속성의 이름은 첫 글자를 소문자로 바꾼 **camelCase** 이고, 대문자로 쓴 약어는 한 단어로 본다.

Table 12. 컬럼과 속성의 이름

컬럼	속성	비고
ItemCode	itemCode	
BOMID	bomId	BOM 과 ID
PONumber	poNumber	PO 와 Number
SODetailID	soDetailId	SO, Detail, ID
Annual Usage	annualUsage	공백이 있는 컬럼

이름에 밑줄이 없으므로 Sakila 나 데모 데이터베이스처럼 snake_case 를 바꿀 일이 없다. SQL Server 는 기본 데이터 정렬(collation)에서 식별자의 대소문자를 가리지 않으므로, @Column(name = "ItemCode") 는 SQL 에서 itemcode 로 써도 같은 컬럼이다. 그래도 이름 상수는 데이터베이스가 정의한 대로 기재한다.

공백이 있는 이름은 vw_InventoryTurnover 의 [Annual Usage] 하나다. SQL Server 는 대괄호로 식별자를 감싸지만, Jakarta Persistence 는 큰따옴표로 감싼 이름을 구분 식별자로 다룬다.

MappedInventoryTurnover 의 annualUsage

```
public static final String COLUMN_NAME_ANNUAL_USAGE = "\"Annual Usage\"";

// 읽기 전용: SQL Server 는 이 뷰에 쓰지 않는다. 클래스의 문서를 보라
@Digits(integer = COLUMN_PRECISION_ANNUAL_USAGE - COLUMN_SCALE_ANNUAL_USAGE,
        fraction = COLUMN_SCALE_ANNUAL_USAGE)
@Basic(optional = true)
@Column(name = COLUMN_NAME_ANNUAL_USAGE,
        nullable = true,
        insertable = false,
        updatable = false,
        precision = COLUMN_PRECISION_ANNUAL_USAGE,
        scale = COLUMN_SCALE_ANNUAL_USAGE
)
private BigDecimal annualUsage;
```

구현체가 이 이름을 큰따옴표 그대로 보내든 대괄호로 바꾸어 보내든 결과는 같다. SQL Server 의 JDBC 드라이버는 QUOTED_IDENTIFIER 를 켜고 연결하므로, 큰따옴표로 감싼 이름도 대괄호로 감싼 이름과 같은 식별자가 된다.

Exercises

이 장의 데이터로 풀 수 있는 연습문제 가운데 이 장의 매핑을 연습하기 좋은 문제를 선택해 모았다. 문제마다 Query Language 와 Criteria API 로 푼 답을 함께 실었다. 풀이는 프로젝트의 `src/test/sql-exercises` 에 있는 SQL 파일과, 같은 문제를 Query Language 와 Criteria API 로 푼 테스트에 있다.

FUTON_DBO_I_02

완제품마다 직접 구성품에 드는 재료비를 손실률까지 더해 구하고, 정가에서 재료비를 뺀 이익을 함께 구한다. 유효한 자재 명세가 없는 완제품은 두 값이 모두 `NULL` 이다.

SQL

```
SELECT p.ItemCode, p.ItemName,
       CAST(SUM(b.Quantity * (1 + b.ScrapRate / 100) * c.StandardCost) AS DECIMAL(18,
           4)) AS MaterialCost,
       p.ListPrice,
       CAST(p.ListPrice - SUM(b.Quantity * (1 + b.ScrapRate / 100) * c.StandardCost)
            AS DECIMAL(18, 4)) AS Margin
FROM Items p
JOIN ItemType t ON t.ItemTypeID = p.ItemTypeID
LEFT JOIN BillOfMaterials b ON b.ParentItemID = p.ItemID AND b.IsActive = 1
LEFT JOIN Items c ON c.ItemID = b.ComponentItemID
WHERE t.TypeCode = 'FG'
GROUP BY p.ItemID, p.ItemCode, p.ItemName, p.ListPrice
ORDER BY p.ItemCode;
```

결과

```
ItemCode    ItemName                              MaterialCost  ListPrice  Margin
FG-FUT-001  Twin Economy Futon - Natural Canvas   314.3468      299.9900   -14.3568
FG-FUT-002  Full Economy Futon - Navy Canvas      423.8052      399.9900   -23.8152
FG-FUT-003  Full Deluxe Futon - Microfiber Black  682.2813      599.9900   -82.2913
FG-FUT-004  Queen Premium Futon - Velvet Emerald  714.5420      799.9900   85.4480
FG-FUT-005  Full Comfort Futon - Chocolate Suede  494.3444      499.9900   5.6456
FG-FUT-006  Queen Luxury Futon - Linen Beige      724.5918      899.9900   175.3982
```

구성품마다 `Quantity` 에 손실률만큼 더한 수량에 구성품의 표준 원가 `StandardCost` 를 곱하고, 완제품마다 더한다. 구성품의 표준 원가에는 그 원자재의 원가가 이미 들어 있으므로, 자재 명세의 1 단계 행만 더한다. FG-FUT-003 의 재료비 682.2813 은 재귀 쿼리 에서 본 `vw_CostRollUp` 의 1364.5630 의 절반쯤이다. 완제품 여섯 가운데 셋은 재료비가 정가보다 커서 이익이 음수다.

자재 명세에서 시작하면 유효한 행이 없는 완제품이 빠지므로, SQL 은 완제품에서 자재 명세로 `LEFT JOIN` 한다. 그런 완제품은 `SUM` 이 `NULL` 이 되고, 정가에서 `NULL` 을 뺀 이익도 `NULL` 이 된다.

Query Language

```
SELECT p, SUM(b.quantity * (1 + b.scrapRate / 100) * c.standardCost),
       p.listPrice - SUM(b.quantity * (1 + b.scrapRate / 100) * c.standardCost)
FROM DboBillOfMaterials b JOIN b.parentItem p JOIN b.componentItem c
WHERE p.itemType.typeCode = :typeCode AND b.isActive = TRUE
GROUP BY p
ORDER BY p.itemCode
```

자재 명세의 한 행에서 `b.parentItem` 으로 부모 품목을, `b.componentItem` 으로 구성품을 조인한다. 두 관계가 모두 `Item` 으로 가므로, 같은 엔티티가 별칭 `p` 와 `c` 로 두 번 조인된다(자재 명세). `SUM` 안에 속성의 산술을 그대로 쓰고, `p.listPrice - SUM(…)` 처럼 집계의 결과로 다시 계산한다. 명세는 `SUM` 의 인자를 상태 필드로 끝나는 경로식으로 정하므로, `SUM` 안의 산술은 Hibernate ORM 의 확장이다.[3] `GROUP BY p` 는 품목 엔티티로 묶는다.

이 답은 SQL 과 달리 자재 명세의 행에서 시작해 내부 조인하므로, 유효한 자재 명세가 없는 완제품은 나오지 않는다. 설치된 데이터에서는 완제품 여섯이 모두 유효한 자재 명세를 가지므로 결과의 행은 SQL 과 같다. 재료비의 `scale` 은 SQL Server 가 정하므로(소수와 이름), 테스트는 소수 넷째 자리로 반올림해 비교한다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var b = query.from(BillOfMaterials.class);
final var p = b.join(BillOfMaterials_.parentItem);
final var c = b.join(BillOfMaterials_.componentItem);
final var cost = builder.sum(builder.prod(
        builder.prod(b.get(BillOfMaterials_.quantity),
                     builder.sum(builder.quot(b.get(BillOfMaterials_.scrapRate), 100), 1)),
        c.get(Item_.standardCost)
));
query.select(builder.array(p, cost, builder.diff(p.get(Item_.listPrice), cost)))
        .where(builder.equal(p.get(Item_.itemType).get(ItemType_.typeCode), "FG"),
               builder.isTrue(b.get(BillOfMaterials_.isActive)))
        .groupBy(p)
        .orderBy(builder.asc(p.get(Item_.itemCode)));
final var list = entityManager.createQuery(query).getResultList();
```

`builder.prod` 는 곱셈, `builder.quot` 는 나눗셈, `builder.diff` 는 뺄셈이다. `builder.sum` 은 인자가 하나이면 집계 함수 `SUM` 이고, 인자가 둘이면 덧셈이다. 집계 식을 변수 `cost` 에 담아 선택 목록에서 두 번 쓴다.

Hibernate ORM 은 `GROUP BY p` 를 부모 품목의 컬럼을 모두 나열한 `group by` 로 바꾼다. `SELECT p` 가 품목의 컬럼을 모두 선택하고, SQL Server 는 집계하지 않고 선택한 컬럼을 모두 `GROUP BY` 에 두어야 하기 때문이다. Query Language 로 보낸 SQL 에는 `ItemTypeID` 와 `UnitID` 가 두 번씩 나온다. 두 컬럼은 매핑된 슈퍼클래스의 읽기 전용 기본 속성과 엔티티의 관계가 함께 매핑하는 외래 키다. Criteria API 로 보낸 SQL 에는 두 컬럼이 한 번씩 나오고, `100` 과 `1` 이 바인딩 매개변수 `?` 로 들어간다.

Hibernate ORM 이 보내는 SQL

```
select pi1_0.ItemID, pi1_0.CreatedDate, pi1_0.Description, pi1_0.IsActive, pi1_0.ItemCode,
       pi1_0.ItemName, pi1_0.ItemTypeID, pi1_0.LeadTimeDays, pi1_0.ListPrice, pi1_0.ModifiedDate,
       pi1_0.ReorderPoint, pi1_0.SafetyStock, pi1_0.StandardCost, pi1_0.UnitID,
       sum(((bom1_0.Quantity*(1+(bom1_0.ScrapRate/100)))*ci1_0.StandardCost)),
       (pi1_0.ListPrice-sum(((bom1_0.Quantity*(1+(bom1_0.ScrapRate/100)))*ci1_0.StandardCost)))
from dbo.BillOfMaterials bom1_0
join dbo.Items pi1_0 on pi1_0.ItemID=bom1_0.ParentItemID
join dbo.ItemType it1_0 on it1_0.ItemTypeID=pi1_0.ItemTypeID
join dbo.Items ci1_0 on ci1_0.ItemID=bom1_0.ComponentItemID
where it1_0.TypeCode=? and bom1_0.IsActive=1
group by pi1_0.ItemID, pi1_0.CreatedDate, pi1_0.Description, pi1_0.IsActive, pi1_0.ItemCode,
         pi1_0.ItemName, pi1_0.ItemTypeID, pi1_0.ItemTypeID, pi1_0.LeadTimeDays, pi1_0.ListPrice,
         pi1_0.ModifiedDate, pi1_0.ReorderPoint, pi1_0.SafetyStock, pi1_0.StandardCost,
         pi1_0.UnitID, pi1_0.UnitID
order by pi1_0.ItemCode
```

FUTON_DBO_I_04

공급자마다 입고할 때 검사한 수량(`InspectionType` 이 `Incoming`) 가운데 불합격한 수량의 비율을, 비율이 높은 순으로 구한다. 비율이 같으면 공급자의 코드순이다. 검사는 구매 주문을 외래 키가 아니라 주문의 번호로 기록하고, 검사한 수량이 0 이면 비율은 `NULL` 이다.

SQL

```
SELECT s.SupplierCode, s.SupplierName, SUM(q.QuantityInspected) AS Inspected,
       SUM(q.QuantityRejected) AS Rejected,
       CAST(SUM(q.QuantityRejected) * 100 / NULLIF(SUM(q.QuantityInspected), 0) AS DECIMAL(9,
           2)) AS RejectedPercent
FROM PurchaseOrder po
JOIN Supplier s ON s.SupplierID = po.SupplierID
JOIN QualityInspection q ON q.ReferenceType = 'PO' AND q.ReferenceNumber = po.PONumber
     AND q.InspectionType = 'Incoming'
GROUP BY s.SupplierID, s.SupplierCode, s.SupplierName
ORDER BY RejectedPercent DESC, s.SupplierCode;
```

결과

SupplierCode	SupplierName	Inspected	Rejected	RejectedPercent
SUP-001	Pacific Textile Mills	100.00	2.00	2.00
SUP-003	Northwest Lumber & Hardware	50.00	1.00	2.00
SUP-005	Luxury Fabric Imports	80.00	1.00	1.25
SUP-002	Premium Fill Supply Co	500.00	5.00	1.00
SUP-004	Industrial Fasteners Inc	200.00	2.00	1.00

검사는 다섯 건뿐이고, 모두 구매 주문을 참조하는 입고 검사다. SUP-001 과 SUP-003 은 비율이 2.00 으로 같으므로 공급자의 코드순으로 놓인다.

`QualityInspection` 의 `ReferenceType` 과 `ReferenceNumber` 를 조인 조건으로 써서 구매 주문과 조인한다(FutonManufacturing). `NULLIF` 는 검사한 수량의 합이 0 일 때 나눗셈 대신 `NULL` 을 돌려준다.

Query Language

```
SELECT s, SUM(q.quantityInspected), SUM(q.quantityRejected),
       SUM(q.quantityRejected) * 100 / SUM(q.quantityInspected) AS rejectedPercent
FROM DboPurchaseOrder po
JOIN po.supplier s
JOIN DboQualityInspection q ON q.referenceType = 'PO' AND q.referenceNumber = po.poNumber
GROUP BY s
ORDER BY rejectedPercent DESC, s.supplierCode
```

검사에는 구매 주문으로 가는 관계가 없으므로, `JOIN DboQualityInspection q ON …` 으로 엔티티를 조인하고 두 조건을 `AND` 로 잇는다. 구매 주문에서 공급자로는 관계 `po.supplier` 를 따라간다. **외래 키로 선언되지 않은 참조는 엔티티에 관계가 없으므로, 엔티티를 직접 조인하고 조인 조건을 기재한다.** 선택 목록에 붙인 별칭 `rejectedPercent` 는 `ORDER BY` 에서 쓴다.

이 답에는 SQL 과 달리 `InspectionType` 의 조건과 `NULLIF` 가 없다. 설치된 데이터의 검사는 모두 입고 검사이고 검사한 수량이 0 인 공급자가 없으므로, 결과는 SQL 과 같다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var po = query.from(PurchaseOrder.class);
final var s = po.join(PurchaseOrder_.supplier);
final var q = po.join(QualityInspection.class);
q.on(builder.equal(q.get(QualityInspection_.referenceType), "PO"),
     builder.equal(q.get(QualityInspection_.referenceNumber), po.get(PurchaseOrder_.poNumber)));
final var inspected = builder.sum(q.get(QualityInspection_.quantityInspected));
final var rejected = builder.sum(q.get(QualityInspection_.quantityRejected));
final var percent = builder.quot(builder.prod(rejected, BigDecimal.valueOf(100)), inspected);
query.select(builder.array(s, inspected, rejected, percent))
        .groupBy(s)
        .orderBy(builder.desc(percent), builder.asc(s.get(Supplier_.supplierCode)));
final var list = entityManager.createQuery(query).getResultList();
```

`po.join(QualityInspection.class)` 로 관계가 아닌 엔티티를 조인하고, `on` 에 조건 둘을 넘긴다. `on` 은 조건을 여러 개 받아 `AND` 로 잇는다. `rejected` 가 `BigDecimal` 의 식이므로 곱하는 100 도 `BigDecimal.valueOf(100)` 으로 넘긴다.

Hibernate ORM 은 두 쿼리 모두 `GROUP BY s` 를 공급자의 컬럼을 모두 나열한 `group by` 로 바꾸고, 비율로 정렬하는 것을 선택 목록의 순서 번호 `18` 로 쓴다. Criteria API 로 보낸 SQL 은 `'PO'` 를 바인딩 매개변수로 보내고, `100` 을 `cast(? as numeric(3, 0))` 으로 보낸다.

Hibernate ORM 이 보내는 SQL

```
select s1_0.SupplierID, s1_0.Address, s1_0.City, s1_0.ContactName, s1_0.Country, s1_0.CreatedDate,
       s1_0.Email, s1_0.IsActive, s1_0.PaymentTerms, s1_0.Phone, s1_0.Rating, s1_0.State,
       s1_0.SupplierCode, s1_0.SupplierName, s1_0.ZipCode, sum(qi1_0.QuantityInspected),
       sum(qi1_0.QuantityRejected),
       ((sum(qi1_0.QuantityRejected)*100)/sum(qi1_0.QuantityInspected))
from dbo.PurchaseOrder po1_0
join dbo.Supplier s1_0 on s1_0.SupplierID=po1_0.SupplierID
join dbo.QualityInspection qi1_0 on qi1_0.ReferenceType='PO'
     and qi1_0.ReferenceNumber=po1_0.PONumber
group by s1_0.SupplierID, s1_0.Address, s1_0.City, s1_0.ContactName, s1_0.Country,
         s1_0.CreatedDate, s1_0.Email, s1_0.IsActive, s1_0.PaymentTerms, s1_0.Phone, s1_0.Rating,
         s1_0.State, s1_0.SupplierCode, s1_0.SupplierName, s1_0.ZipCode
order by 18 desc, s1_0.SupplierCode
```

[1] Microsoft. SQL Server Samples, samples/databases/futon-manufacturing. https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/futon-manufacturing

[2] Microsoft. SQL Server Samples, LICENSE. 저장소의 예제는 MIT 라이선스로 배포된다.

https://github.com/microsoft/sql-server-samples/blob/master/license.txt

[3] Jakarta Persistence 3.2, 4.9.5 Aggregate Functions in the SELECT Clause. "For all aggregate functions except COUNT, the path expression that is the argument to the aggregate function must terminate in a state field." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a5538